

Day 27: Zephyr Threads & Scheduling

Platform: nRF52840 | **RTOS:** Zephyr RTOS

Learning Objectives

K_THREAD_DEFINE

The easiest way to create a Zephyr thread at compile time:

```
K_THREAD_DEFINE(  
    sensor_tid,          // Thread ID variable name  
    1024,                // Stack size in bytes  
    sensor_thread,       // Thread function  
    NULL, NULL, NULL,    // Parameters (arg1, arg2, arg3)  
    5,                  // Priority (lower = higher priority)  
    0,                  // Options  
    0                    // Delay before start (0 = immediate)  
);
```

The thread starts automatically when Zephyr boots (before `main()`).

k_thread_create

For dynamic thread creation (at runtime):

```
K_THREAD_STACK_DEFINE(worker_stack, 512);  
struct k_thread worker_thread;  
  
k_thread_create(&worker_thread, worker_stack, K_THREAD_STACK_SIZEOF(worker_stack),  
               worker_func, arg1, arg2, arg3,  
               priority, 0, K_NO_WAIT);  
k_thread_name_set(&worker_thread, "worker");
```

Thread Priorities (Cooperative vs Preemptive)

Zephyr has two scheduling modes:

- **Preemptive threads** (priority 0-14): can be interrupted by higher-priority threads mid-execution
- **Cooperative threads** (priority -1 to -CONFIG_NUM_COOP_PRIORITIES): run until they voluntarily yield (call `k_yield()`, `k_msleep()`, or block on a sync primitive)

Lower priority number = higher priority. `main()` runs at priority 0 by default in most Zephyr configurations.

k_sleep / k_msleep

`k_msleep(N)` suspends the current thread for N milliseconds and lets other threads run:

```
k_msleep(100);           // Sleep 100ms
k_sleep(K_MSEC(100));    // Equivalent
k_sleep(K_SECONDS(5));   // Sleep 5 seconds
k_sleep(K_FOREVER);      // Sleep indefinitely (until woken)
```

k_yield

`k_yield()` gives other threads of the **same priority** a chance to run:

```
while (processing) {
    do_some_work();
    k_yield();    // Let same-priority threads run
}
```

Higher-priority threads preempt automatically — `k_yield` is only needed for cooperative fairness among equal-priority threads.

Real-time Constraints & WCET

Worst-Case Execution Time (WCET) is the maximum time a code path can take. For real-time systems:

- A 1kHz sensor ISR must complete in < 1ms total (including all interrupts at the same priority)
- A 10ms control loop thread must complete within 10ms

- Use DWT cycle counter to measure WCET during development

Thread Stack Analysis

Stack overflow is a common embedded bug. Zephyr provides stack usage analysis:

```
CONFIG_STACK_SENTINEL=y      # Detect overflow (adds canary)
CONFIG_THREAD_STACK_INFO=y   # Enable stack usage reporting
CONFIG_THREAD_ANALYZER=y     # Periodic stack usage report
```

Run `kernel stacks` command in Zephyr shell to see peak stack usage for all threads.

Practice Tasks

1. Create 3 threads at different priorities — verify higher priority threads preempt lower ones
 2. Use `CONFIG_THREAD_ANALYZER=y` to find the peak stack usage of each thread — right-size the stacks
 3. Measure how long a thread context switch takes using DWT cycle counter
 4. Create a cooperative thread that calls `k_yield()` — verify it shares CPU fairly with a same-priority thread
-

Build & Run

```
cd /home/eva/workspace/My-CPP_learning/src/day27
west build -b nrf52840dk/nrf52840 .
west flash
# View output via RTT or UART serial (115200 baud)
west attach # RTT viewer (requires J-Link)
```

Resources

- [Zephyr Docs](#)
- [nRF52840 Product Specification](#)
- [Nordic DevZone](#)
- Source: [src/day27/main.cpp](#)